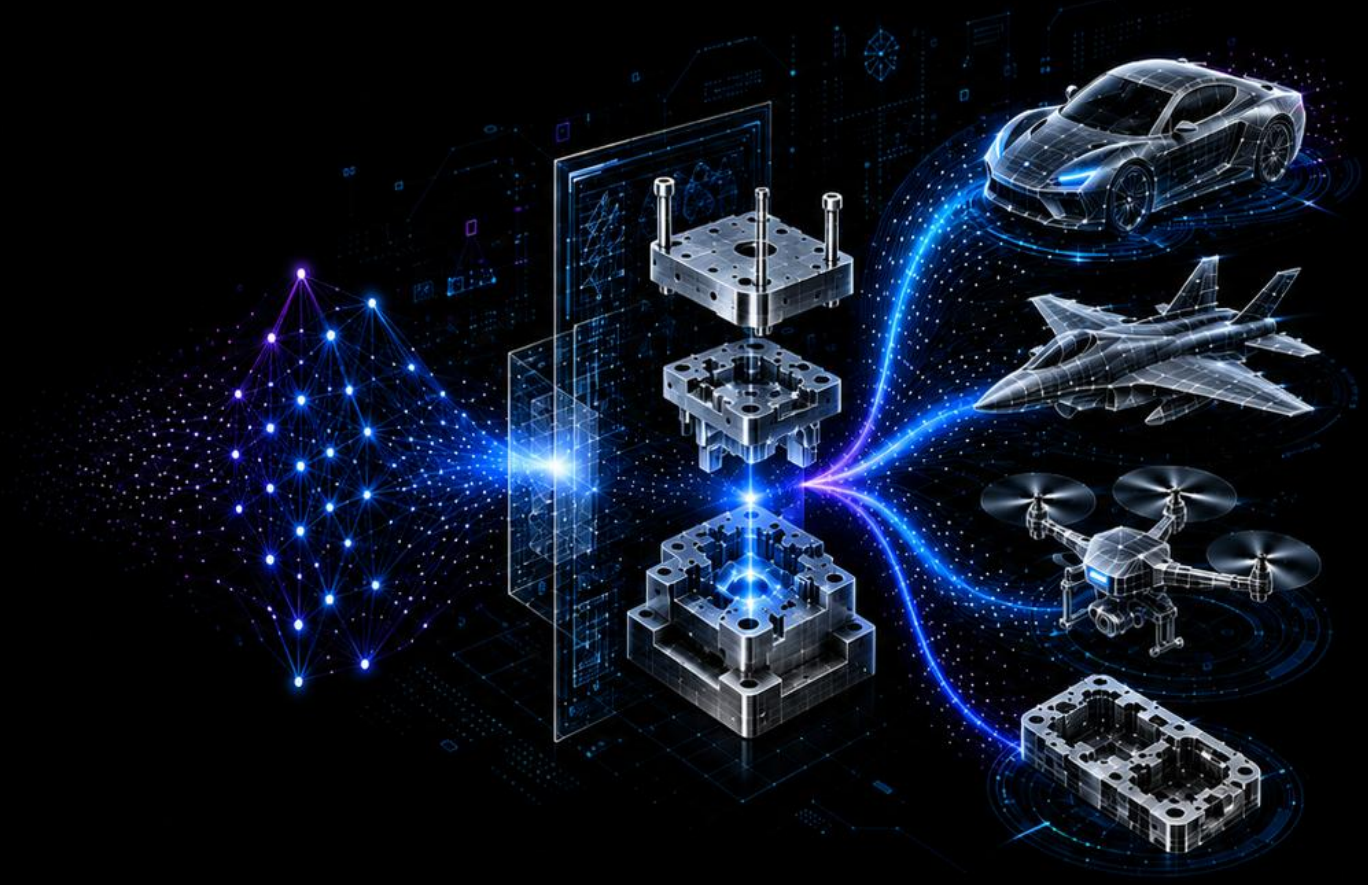
Intern Task

Adaptive Table-aware Path-KG RAG & LLM WIKI



신동우

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과

Intelligent Design for Engineering Applications Lab

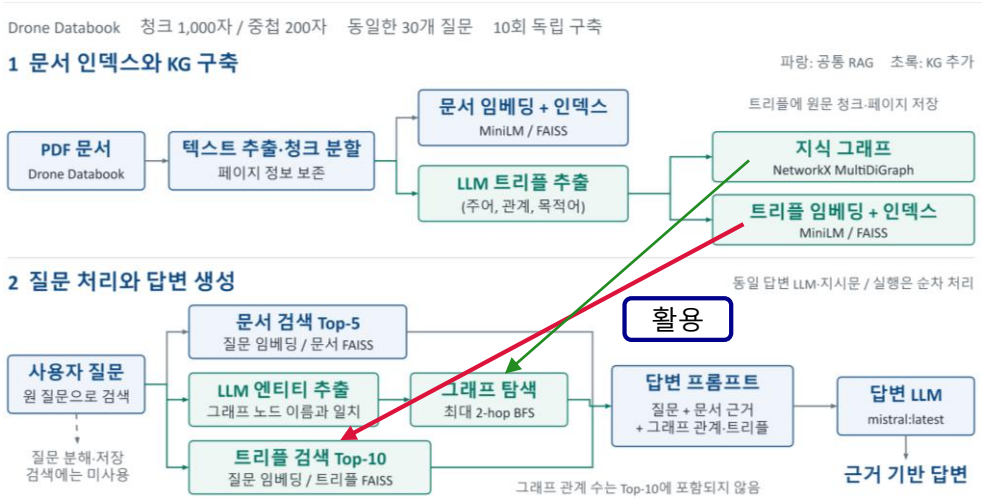
Adaptive Table-aware Path-KG RAG

이전 실험한 KG-RAG의 한계

- KG를 추가했다고 항상 좋아지지 않는다

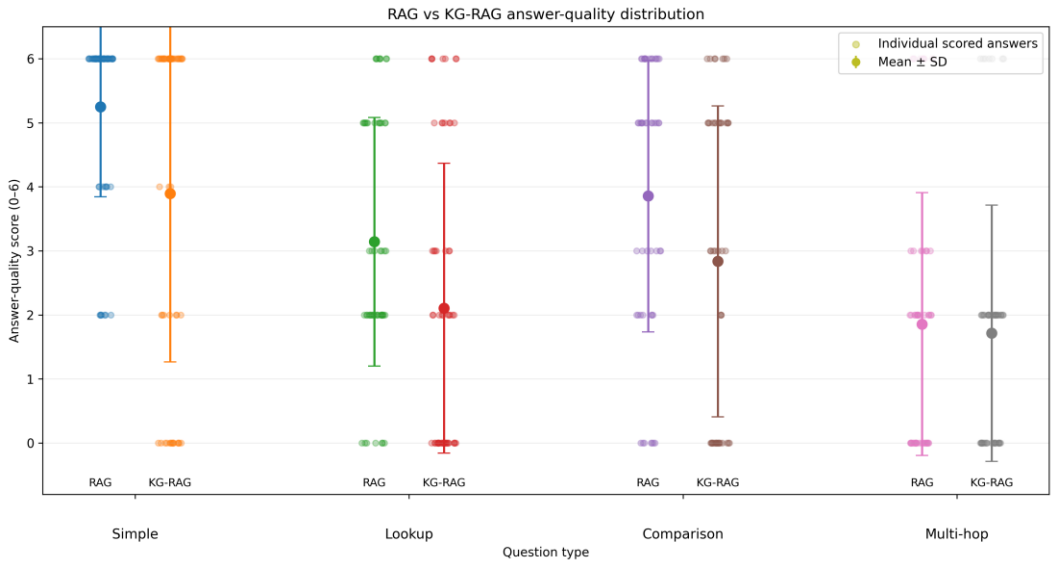
기존 framework

❖ 초록색과 빨간색 화살표는 화살표 방향으로 활용된다는 의미



figure

Type	RAG	KG-RAG
Simple	**5.25**	3.89
Lockup	**3.14**	2.11
Comparison	**3.86**	2.84
Multi-hop	**1.86**	1.71



Adaptive Table-aware Path-KG RAG

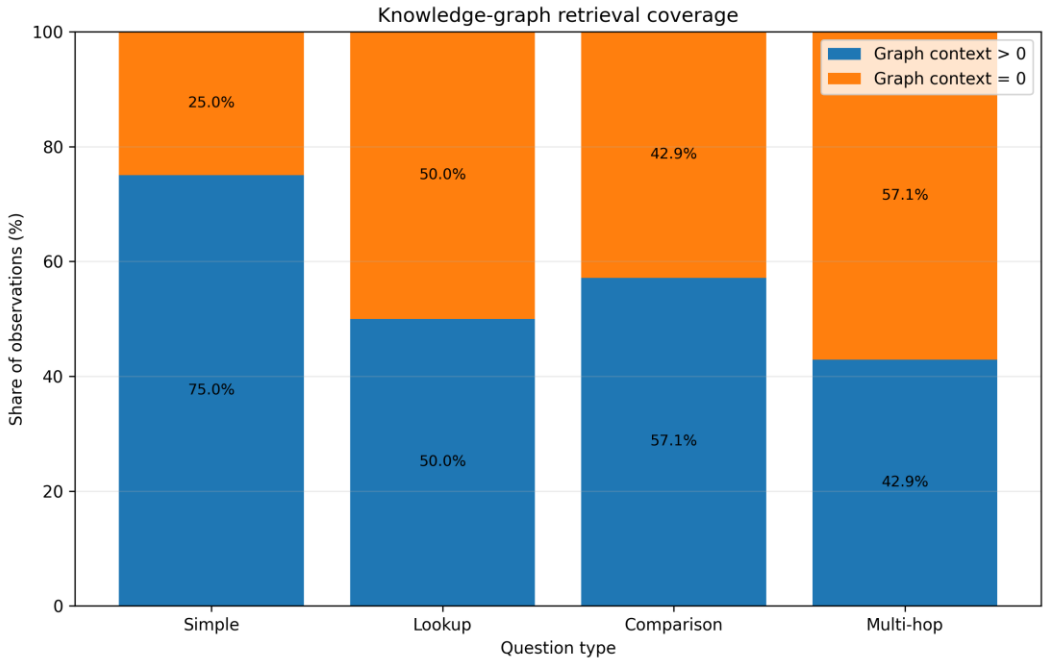
Intelligent Design for Engineering Applications Lab

이전 실험한 KG-RAG의 한계

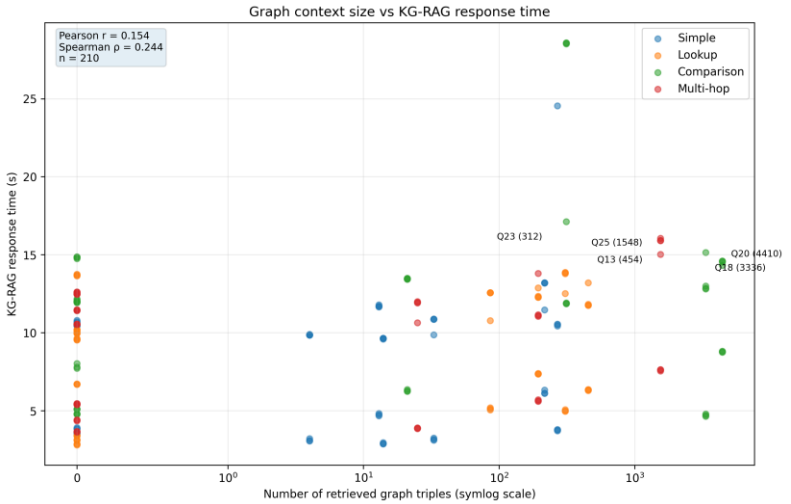
- 왜 기존 KG-RAG가 실패했는가

Graph Retrieval Failure

❖ C-Astral, C-ASTRAL처럼 표기 차이나 Entity matching 실패가 있으면 KG가 있어도 실제 retrieval에서는 활용되지 않는 문제 발생



Context Pollution



Q/A Case

Q/A Case 3 — KG-RAG가 오히려 틀리는 경우

Question

Which UAVs are manufactured by C-Astral?

RAG

> The retrieved evidence does not contain enough information to determine which UAVs are manufactured by C-Astral.

KG-RAG

> C-Astral manufactures the CubicFlyer UAV.

Adaptive Table-aware Path-KG RAG

전체 실험 구조

실험 조건

Original Baseline

Chunk : Fixed Chunk
Embedding : BGE M3
KG: : Current KG Retrieval
Table : Plain Text
LLM : Mistral 7B

1. 단계별로 총 3번의 반복 과정을 수행한다.

2. 새로운 질문을 추가로 설정하여 (1)"테이블에 관한 내용의 LLM 응답"과 (2)"직접 답이 없는 요구조건을 근거 기반으로 판단하는가"를 추가

단계	비교 조건	나머지 조건	선택
A	Fixed vs Adaptive Chunk	MiniLM + Current KG + Plain Table + Mistral	더 좋은 Chunk 선택
B	~~MiniLM~~ vs Qwen 3 Embedding vs BGE M3	**A** 에서 선택된 **Chunk** + Current KG + Plain Table	Embedding 선택
C	Current KG vs Seed + Path-pruned KG	**A + B** 최적 조건 + Plain Table + Mistral	KG 방식 선택
D	Plain Text Table vs Structured Table	**A + B + C** 최적조건 + Mistral	Table 방식 선택
E	Mistral 7B vs Qwen3 8B	**A + B + C + D** 최적 조건	LLM 선택

MiniLM은 256 token 제한으로 사용하지 않는 것으로 바꿈

전체 실험 구조

실험과 평가 지표

실험	가장 중요한 평가 지표
A Chunk	Recall@K, Context Precision, Triple F1, Answer Quality
B Embedding	Recall@K, MRR/nDCG, KO/EN retrieval, Answer Quality
C KG Retrieval	Multi-hop Success, Path Precision, Retrieved Triple 수, Context Pollution, Answer Quality
D Table	Table Hit@K, Cell/Row Accuracy, Numerical Accuracy, Design Reasoning
E LLM	Triple Precision/Recall/F1, Correctness, Completeness, Grounding, Reasoning Quality

Adaptive Table-aware Path-KG RAG

Experiment A on Chunk

- Fixed vs Semantic Chunking

연구 사례: "*SC-LKM: Asemantic Chunking and Large Language Model-Based Cybersecurity Knowledge Graph Construction Method", Electronics, 2025

Comparison

아래 Fig 1처럼 Experiment A에서는 기존 Chunk 1200/200를 사용하고, 새로운 방식으로 Adaptive Chunk를 사용한다

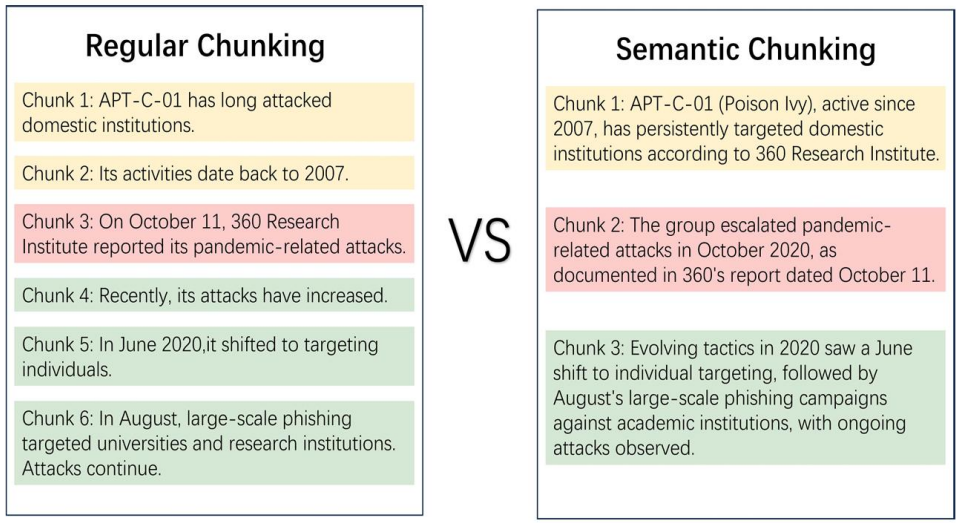


Figure 1. Comparison diagram of regular chunking and semantic chunking examples.

*SC-LKM ≈ **S**emantic **C**hunking + **L**LM-based **K**nowledge Graph Construction **M**ethod

Adaptive Chunk

❖ SC-LKM 논문에서 핵심 논리 4개를 가져온다

① **Document Structure**

○ AUSTRALIA
- ◇ Personnel
- ◇ Inventory
  - →Active Inventory
  - →Inactive Inventory
- ◇ Infrastructure

→ Inventory → Infrastructure로 section이 바뀐다면

$$H_i = \begin{cases} 1, & \text{heading/section changes} \\ 0, & \text{otherwise} \end{cases}$$

Inventory -〉 Inventory
H = 0

Inventory -〉 Infrastructure
H = 1

H=1일 때 강제로 Split하도록 구현

D는 Document Structure로 원문 문서를 Chapter -> Section -> Paragraph 구조로 나눈 기본 단위들

$$D = \{P_1, P_2, \cdots, P_n\}$$

② **Topic Similarity**

현재 Chunk c 와 새 paragraph P_i 를 MiniLM(embedding model)에 넣는다

f는 imbedding model $f(c) = \mathrm{v}_c \quad f(P_i) = \mathrm{v}_i \longrightarrow s_i = \cos(\mathrm{v}_i, \mathrm{v}_c)$

$s_i < \theta_s$, θ_s는 임계값

임계값보다 높으면 Chunk에 붙일 후보로 선정

Adaptive Table-aware Path-KG RAG

Experiment A on Chunk

- Fixed vs Semantic Chunking

연구 사례: "*SC-LKM: Asemantic Chunking and Large Language Model-Based Cybersecurity Knowledge Graph Construction Method", Electronics, 2025

Adaptive Chunk

③ **Entity Continuity**
Ex) Paragraph:
➔ C-Astral manufactures the Bramor UAV.

Entity:

$$E_i = \{C - Astral, Bramor\}$$

현재 Chunk에 이미 존재하는 Entity가:

$$E_c = \{Bramor, New\ Zealand\}$$

라고 하면
겹치는 Entity는:

$$E_i \cap E_c = \{Bramor\}$$

여기서 overlap()을 어떻게 정의할지 구현하는 선택은 가장 단순한 방법 중 하나는 Jaccard similarity

$$e_i = \frac{|E_i \cap E_c|}{|E_i \cup E_c|}$$

*SC-LKM ≈ **S**emantic **C**hunking + **L**LM-based **K**nowledge Graph Construction **M**ethod

Adaptive Chunk

$$E_i \cup E_c = \{C - Astral, Baramor, New\ Zealand\}$$

이므로 따라서:

$$e_i = \frac{1}{3} = 0.333$$

또 다른 구현은 새 paragraph의 Entity 중 기존 Chunk에서 얼마나 이어졌는지만 보는 것

$$e_i = \frac{|E_i \cap E_c|}{|E_i|} \qquad e_i < \theta_s$$

➔ 새 paragraph에서 등장한 Entity가 지금까지 설명하던 Entity와 얼마나 이어지는가?

④ **Length Constraint**
현재 Chunk 길이가 L 이라고 하고 새 paragraph가 l_i 라고 하면

$$L + l_i > L_{\max}$$ 인지 확인한다.

즉
Current Chunk = 460 tokens
New Paragraph = 90 tokens
Maximum = 512 tokens
이므로 **SPLIT**

$$460 + 90 = 550$$
$$550 > 512$$

Experiment A에서 ※임계값은 데이터와 목적에 맞게 검증해서 정하는 hyper-parameter

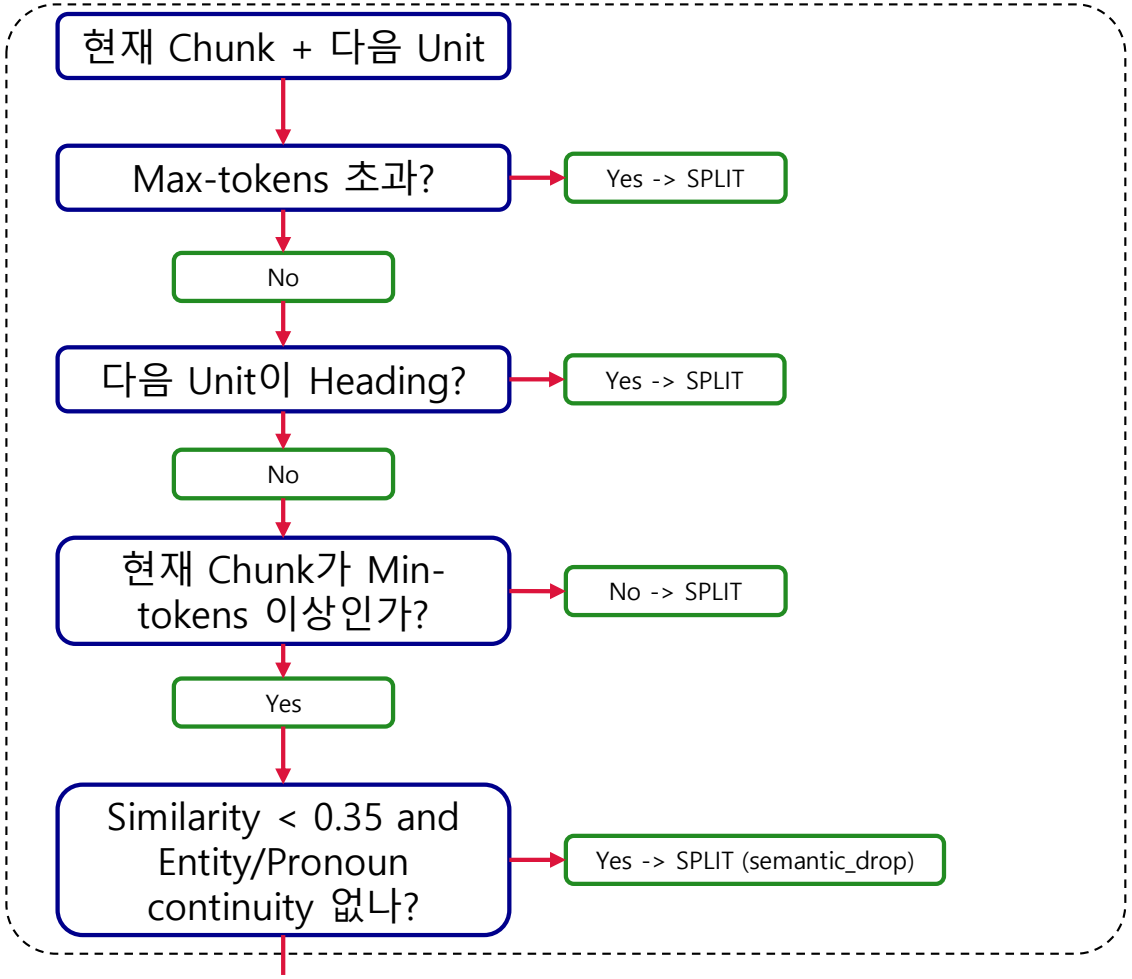
$$\boxed{H_i = 1 \text{ OR } s_i < \theta_s \text{ OR } e_i < \theta_s \text{ OR } L + l_i > L_{\max}}$$

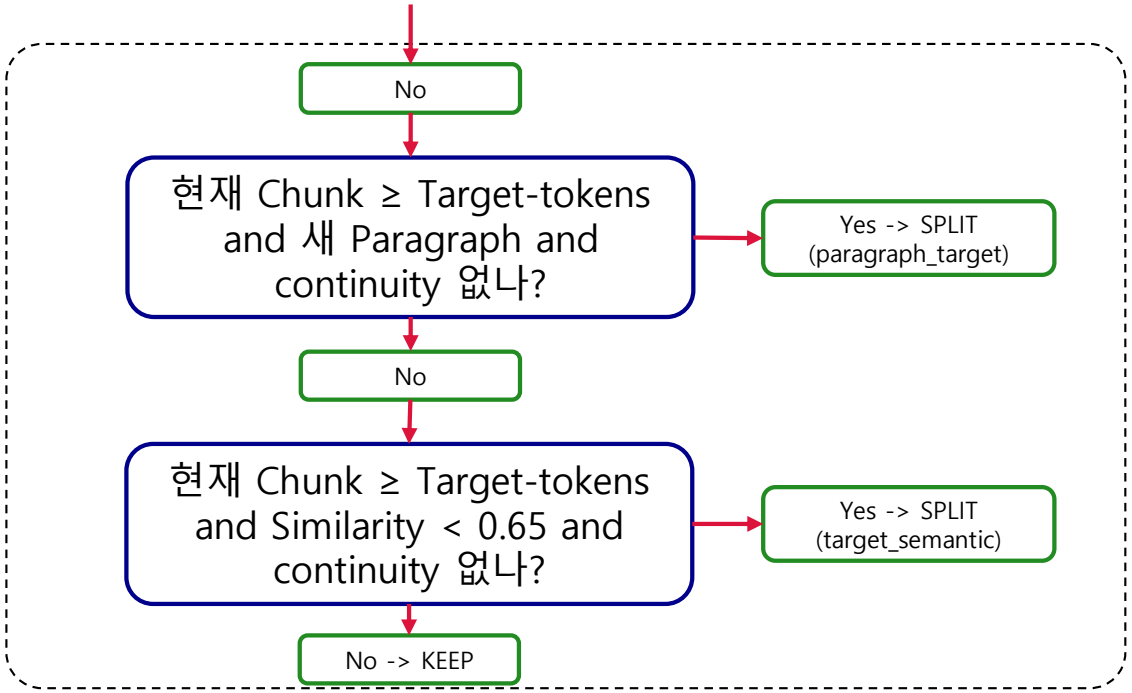
이면 판단하여 Split이라고 표현할 수 있다.

Adaptive Table-aware Path-KG RAG

Experiment A on Chunk

- Experiment A에 적용된 판단 순서
- Code Test





- ❖ Pdf는 논문 하나를 예시로 사용한다.
  → (drone databook은 page가 너무 길어서 간단한 실험으로 사용해서 판단하기 힘들기 때문)
- ❖ Chunk가 제대로 되었는지 txt 파일로 출력하여 직접 판단
- ❖ 제대로 Chunking되지 않았다면 개선점 모색하기

Adaptive Table-aware Path-KG RAG

Experiment A on Chunk

- Result of Chunking process

결과 분석

❖ 현재 방식에 문제가 많아 개선해야 한다

단계	현재 방식
PDF 처리	PDF line 유지
Sum-\nmarization	그대로 분리
Unit	PDF line 또는 line 일부
Structure	간단한 heading heuristic
Similarity	이전 unit ↔ 현재 unit
문법 관계	고려하지 않음
Entity	대문자 regex
Page end	무조건 SPLIT
Token 초과	바로 split
Chunk 관계	독립
Table	일반 text 와 혼합

Chunking 결과

Chunk 8, Chunk 9 →

Unit의 KEEP ↓

```
##################################################################################
UNIT 46 → KEEP
##################################################################################

[CURRENT UNIT]
intuitive natural language-based commands, enabling not only

[PREVIOUS UNIT]
and DeepSeek have emerged as intelligent tools that allow

[CURRENT CHUNK BEFORE ADDING UNIT 46]
------------------------------  1. Introduction  The rapid advancement of large language models (LLMs)  has expanded their applications beyond text generation,
sum- marization, and translation to encompass complex reasoning  and decision-making tasks. Models such as ChatGPT, Gemini,  and DeepSeek have emerged as
intelligent tools that allow

--- STRUCTURE FLAGS ---
heading             = False
paragraph           = False
forced_fragment     = False

--- LENGTH ---
tokens_before       = 91
tokens_if_joined    = 101
MIN_TOKENS          = 96
TARGET_TOKENS       = 192
MAX_TOKENS          = 254

--- TOPIC SIMILARITY ---
similarity(prev,current) = 0.27568650245666504
SIMILARITY_THRESHOLD     = 0.35
TARGET_SIMILARITY        = 0.65

--- ENTITY CONTINUITY ---
current unit entities    = []
preceding entities       = ['chatgpt', 'deepseek', 'gemini', 'introduction', 'llms', 'models']
entity_jaccard           = 0.0000
pronoun_continuity       = False
continuous               = False

--- DECISION CHECK ORDER ---
1) total > MAX_TOKENS ? 101 > 254 → False
2) heading ? False
3) before >= MIN_TOKENS ? 91 >= 96 → False

>>> FINAL DECISION
>>> KEEP
>>> reason = keep
>>> decision_time = 0.000689 sec
```

```
--------------------------------------------------------------------------------
CHUNK 8
--------------------------------------------------------------------------------
page        : 1
chars       : 408
tokens      : 101
reason      : semantic_drop
below_min   : False
char range : 3028 ~ 3436

[TEXT]
------------------------------

1. Introduction
The rapid advancement of large language models (LLMs)
has expanded their applications beyond text generation, sum-
marization, and translation to encompass complex reasoning
and decision-making tasks. Models such as ChatGPT, Gemini,
and DeepSeek have emerged as intelligent tools that allow
intuitive natural language-based commands, enabling not only

--------------------------------------------------------------------------------
CHUNK 9
--------------------------------------------------------------------------------
page        : 1
chars       : 565
tokens      : 99
reason      : semantic_drop
below_min   : False
char range : 3438 ~ 4003

[TEXT]
experts but also non-experts to perform sophisticated opera-
tions with ease. Consequently, LLM-based systems are in-
creasingly being adopted in domains that traditionally required
specialized expertise, bringing significant transformations
across industries such as manufacturing, construction, and
engineering design [1].
In engineering applications, LLMs have evolved beyond nat-
ural language processing. They now enable the autonomous
generation and execution of software commands and can also
control physical hardware such as robotic arms, drones, and
```

*adaptive_chunk_debug_results_1: 결과 폴더 이름

Adaptive Table-aware Path-KG RAG

Experiment A on Chunk

- 개선된 Chunking_v2

결과 분석

❖ Page로만 section을 나눴던 것을 대제목, 소제목으로도 section을 나눈다.

❖ Section으로 나눈 결과 같은 section 안에 있는 글을 이어지게 할 수 있다.

❖ Max tokens로 Split을 진행해도 문장 중간에 자르는 것이 아닌 문장의 끝인 마침표가 있는 부분을 기준으로 split

❖ 이전 버전에서 끊어져 있어서 문맥이 이어지지 않았던 것을 하나의 Chunk 안으로 모아서 문맥이 끊기지 않게 바꿈

Chunk 결과

CHUNK 2

tokens : 215
chars : 1155
pages : [1]
section_path : ['Abstract']
reason : max_tokens_safe_sentence_boundary
below_min : False
prev_chunk_id : 1
next_chunk_id : 3
overlap_sent. : 0

[TEXT]
Abstract

This study proposes an integrated LLM–Abaqus platform that links u
modification and remodeling of analysis conditions through natural la
models via image preprocessing and multi-similarity evaluation using
automates preprocessing, computation, and postprocessing procedu
(AST) analysis and retrieval-augmented generation (RAG) techniques
allowing efficient handling of complex parameters such as load cond
validated through four scenarios: simple image input, basic analysis c

이어진다

CHUNK 3

tokens : 158
chars : 890
pages : [1]
section_path : ['Abstract']
reason : section_boundary
below_min : False
prev_chunk_id : 2
next_chunk_id : 4
overlap_sent. : 1

같은 section_path

[TEXT]
Abstract

The performance of the proposed framework was validated through four scenarios: simple image input, basic analysis condition modification, time-dependent load variation, and multi-command composite input. In all cases, the LLM-based automation achieved results equivalent in accuracy to conventional GUI-based methods, while reducing the processing time to 28–43% of the manual approach during iterative design updates. This study provides a foundation for enhancing the efficiency of structural analysis in educational, research, and industrial applications by enabling intuitive control and full automation of the FEA process. While the current validation focuses on fundamental beam structures to establish the core workflow, the proposed multi-similarity image-matching framework is designed to be extensible to more complex 3D geometries as the image–script database expands.

CHUNK 4

tokens : 240
chars : 1396
pages : [1]
section_path : ['1. Introduction']
reason : max_tokens_safe_sentence_boundary
below_min : False
prev_chunk_id : 3
next_chunk_id : 5
overlap_sent. : 0

Max tokens로 split해도 중간 단어 사이에서 끊기는 것이 아닌 문장으로 끊긴다.(마침표)

[TEXT]
1. Introduction

The rapid advancement of large language models (LLMs) has expanded their applications beyond text generation, summarization, and translation to encompass complex reasoning and decision-making tasks. Models such as ChatGPT, Gemini, and DeepSeek have emerged as intelligent tools that allow intuitive natural language-based commands, enabling not only experts but also non-experts to perform sophisticated operations with ease. Consequently, LLM-based systems are increasingly being adopted in domains that traditionally required specialized expertise, bringing significant transformations across industries such as manufacturing, construction, and engineering design [1]. In engineering applications, LLMs have evolved beyond natural language processing. They now enable the autonomous generation and execution of software commands and can also control physical hardware such as robotic arms, drones, and 3D printers. For instance, several open-source projects have demonstrated systems that convert natural language commands into executable Python code to control hardware devices. Frameworks such as AutoGPT and LangChain have further enabled automated problem-solving by integrating external software and databases [2–4]. These technologies have effectively lowered the entry barrier for non-specialists and enhanced responsiveness across a wide range of operational scenarios.

Adaptive Table-aware Path-KG RAG

Experiment A on Chunk

- 개선된 Chunking_v2

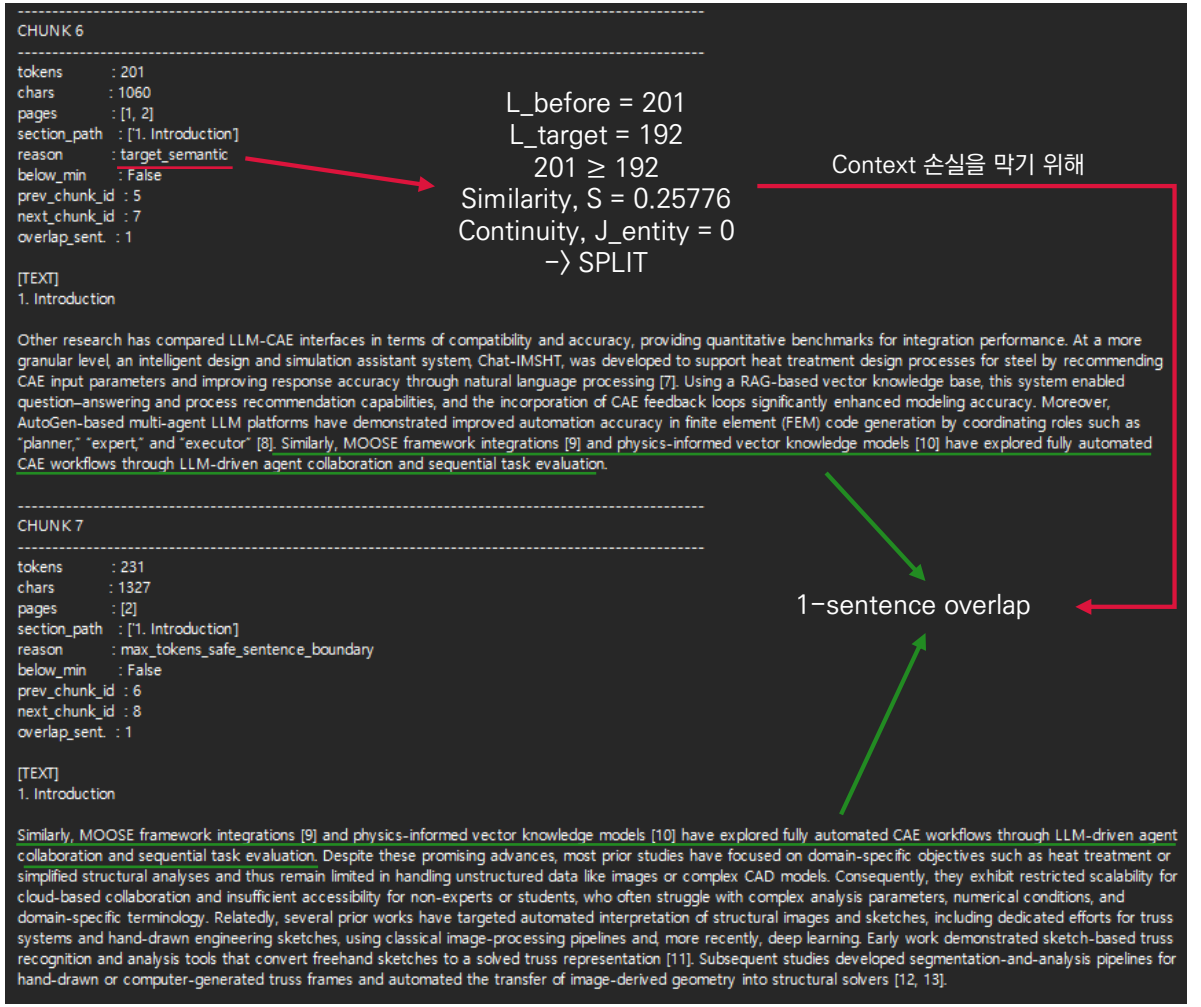
결과 분석

❖ 임계값을 기준으로 split을 하였을 때 글 전체의 문맥이 끊어질 수도 있다.

❖ 문맥의 흐름이 끊어지는 것을 막고 전 내용을 참고하기 위해서 overlap 사용

❖ Overlap으로 context 손실을 막을 수 있다.

Chunk 결과



Adaptive Table-aware Path-KG RAG

Experiment A on Chunk

- 개선된 Chunking_v2와 초기 버전의 구체적인 차이

❖

항목	초기 버전 Adaptive_chunk_debug.py	v2.1.1	의미
기본 입력 단위	PDF line	복원된 paragraph/sentence	PDF 레이아웃 영향을 줄임
Page 경계	강제 chunk 경계	metadata 만 유지	페이지를 넘는 문장 허용
Header/Footer	그대로 들어올 수 있음	반복 margin 제거	불필요한 검색 noise 감소
Hyphen	없음	dehyphenation	reason-ing -> reasoning
실제 compound hyphen	별도 처리 없음	whitelist 보존	Retrieval-augmented 유지
Heading	단순 line heuristic	계층형 heading	2 -> 2.3 -> … 구조 보존
여러 줄 heading	처리 어려움	연결하여 복원	긴 section 제목 정상화
Sentence	line 안에서만 분리	paragraph 복원 후 분리	문장 중간 절단 감소
Similarity	이전 unit ↔ 현재 unit	현재 chunk context ↔ 다음 context	topic 변화 판단 개선
Threshold	항상 0.35 중심	Local dynamic threshold 추가	문서 구간 특성 반영
Semantic split	MIN 이후 어디서나 가능	paragraph/complete sentence 경계 중심	과도한 split 감소
Heading chunk	작은 chunk 발생 가능	Metadata/prefix	heading 단독 chunk 감소
Overlap	없음	1 sentence overlap	경계 정보 손실 완화
Page provenance	chunk 당 한 page	pages = […]	multi-page chunk 추적
Prev/next	없음	prev_chunk_id, next_chunk_id	Chunk 관계 추적

Adaptive Table-aware Path-KG RAG

Context + Chunk

- Introducing Contextual Retrieval

"For an AI model to be useful in specific contexts, it often needs access to background knowledge.", 2024

기존 RAG와의 차이

❖ Contextual Retrieval 방법: 각 Chunk에 원문 전체에서의 위치와 의미를 설명하는 짧은 문맥을 먼저 붙인 뒤 검색용 데이터로 저장

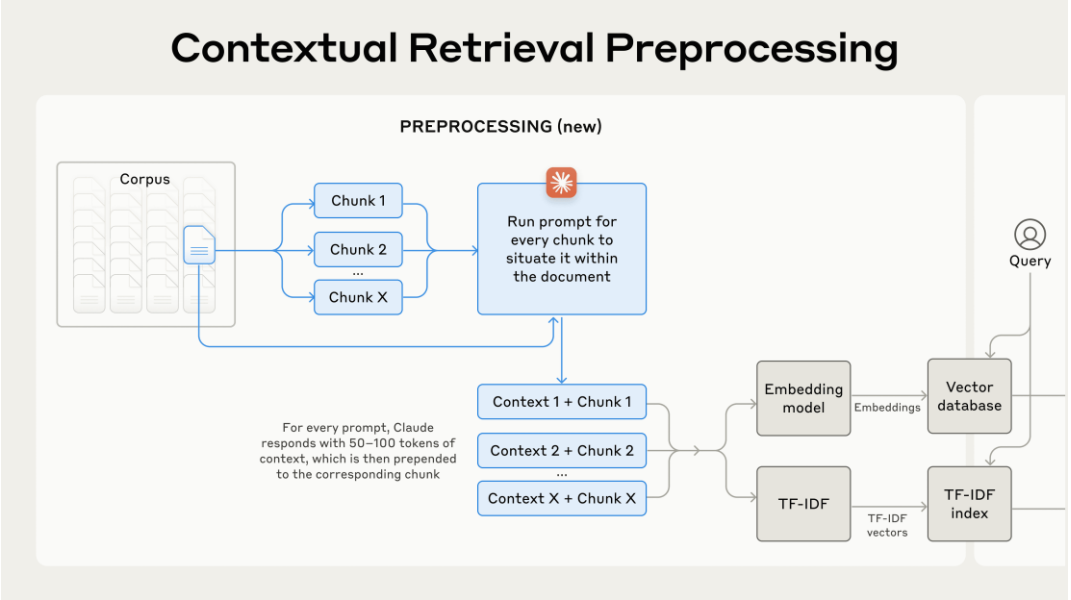
❖ 기존 RAG

Document → Chunking → Embedding → Vector DB → Query → Top-K Retrieval → LLM

❖ Contextual Retrieval

Document → Chunking → **각 Chunk에 문맥 설명 생성** → **[Context + Chunk]** → **Embedding/BM25** → Vector DB → Query → Top-K Retrieval → LLM

figure



Adaptive Table-aware Path-KG RAG

Context + Chunk

- Anthropic의 해결 방법: Chunk 앞에 Context를 붙이기
- Contextual Embedding과 BM25

Contextualized Chunk

❖ 기존 Chunk

```
The company's revenue grew by 3% over the previous quarter.
```

❖ Contextualized Chunk

```
This chunk is from an SEC filing on ACME Corp's
performance in Q2 2023; the previous quarter's
revenue was $314 million.

The company's revenue grew by 3% over the previous quarter.
```

Contextualized Chunk = Context + Original Chunk

LLM이 생성

❖ 중요한 점: Context를 임의로 만드는 것이 아니라 LLM이 전체 문서를 읽고 해당 Chunk의 위치와 의미를 판단해서 생성하는 것

Contextual Embedding + BM25

❖ 기존 Embedding과의 차이

$$E_i = f(C_i) \quad \Leftrightarrow \quad E_i = f(\text{Context}_i + C_i)$$

➔ 기존 Embedding과 다르게 Context도 같이 포함하여 Embedding을 진행

```
Chunk
"The company's revenue grew by 3%..."
        ↓
Embedding Model
        ↓
[0.23, -0.51, 0.12, ...]
```

```
ACME Corp / Q2 2023 / SEC filing
+
The company's revenue grew by 3%...
                ↓
        Embedding Model
                ↓
      [0.41, -0.32, ...]
```

❖ 같이 Embedding함으로써 Context의 의미가 추가되어

```
ACME Corp Q2 2023 revenue growth
```

라고 질문을 했을 때 query embedding과 더 가까워질 가능성이 커지는 것

❖ BM25를 Embedding과 같이 사용하는 이유

방법	잘 찾는 것
Embedding	의미적으로 비슷한 문장
BM25	정확한 단어, 모델명, 코드, 고유명사
두 방법 결합	의미 + 정확한 표현

Adaptive Table-aware Path-KG RAG

Context + Chunk

- BM25는 무엇인가
- BM25의 작동 원리와 계산

❖ BM25란 정보 검색에서 가장 널리 사용되는 **키워드 기반 검색 알고리즘**, Okapi BM25 시스템에서 유래

❖ Vector Search(FAISS) ↔ BM25(키워드 기반 검색)

Vector Search →

```
# 의미 기반 검색 예시
query = "강아지가 좋아하는 음식"
# → 임베딩 벡터로 변환
# → 코사인 유사도로 검색

결과: "반려동물 사료", "개 간식", "펫푸드" 등
# 직접적인 키워드가 없어도 의미가 유사한 문서 검색 가능
```

BM25 →

```
# 키워드 기반 검색 예시
query = "강아지가 좋아하는 음식"
# → 단어 분리: ["강아지", "좋아하는", "음식"]
# → 각 단어의 출현 빈도 계산

결과: "강아지"와 "음식"이 많이 등장하는 문서들
# 정확한 키워드 매칭에 강함
```

❖ **용어 빈도(Term Frequency, TF)**: 특정 단어가 문서 내에서 등장하는 빈도
-〉 높을수록 해당 문서가 그 단어와 관련이 깊다고 볼 수 있다. 단순 빈도수치만으로는 과대 평가가 될 위험이 있기에 BM25가 조절

❖ **문서 빈도(Document Frequency, DF)**: 특정 단어가 몇 개의 문서에서 등장하는지 나타내는 값
-〉 이를 통해 단어의 희귀성을 평가

❖ **역문서 빈도(Inverse Document Frequency, IDF)**: DF의 역수를 취해 계산
-〉 일반적인 단어보다 희귀한 단어가 더 높은 가중치를 받는다.

❖ BM25의 계산식

$$\text{Score} = IDF(q_i) \cdot \frac{f(q_i,D)\cdot(k_1+1)}{f(q_i,D)+k_1\cdot(1-b+b\cdot\frac{|D|}{\text{avgdl}})}$$

$IDF(q_i)$: 쿼리의 i번째 단어에 대한 역문서 빈도 $f(q_i, D)$: 문서 D내에서 쿼리의 i번째 단어의 빈도

$|D|$: 문서 D의 길이(단어 수) avgdl: 문서 집합의 평균 문서 길이

k_1, b: 자유 매개변수로, 각각 용어 빈도와 문서 길이의 영향력을 조절. 일반적으로 $k_1 = 2.0, b = 2.0$

❖ 계산 동작 단계

1. 용어 빈도 조정: 문서 내의 각 단어에 대해 용어 빈도를 계산하고, 위 식에 따라 조정
2. 역문서 빈도 계산: 쿼리를 구성하는 각 단어에 대해 역문서 빈도를 계산. 희귀 단어일수록 높은 값을 가짐
3. 문서 길이 조정: 문서의 길이가 평균보다 길거나 짧은 경우, 이에 따른 조정을 식에 반영
4. 최종 점수 계산: 위에서 계산된 값들을 합산하여 최종 점수를 도출합니다. 이 점수를 기반으로 문서들을 정렬

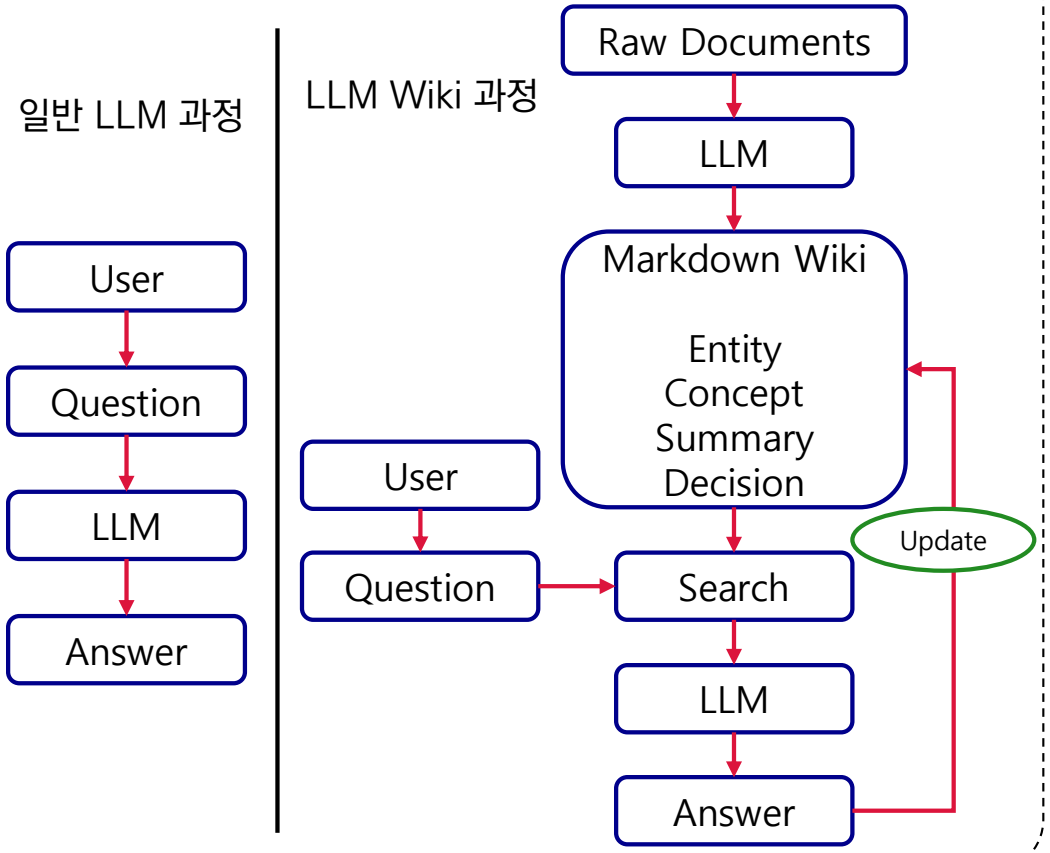
LLM WIKI

What is LLM Wiki

- LLM Wiki 구조와 동작

일반 LLM과 LLM Wiki

LLM Wiki = LLM이 기억하는 것이 아니라 LLM이 외부 Markdown 지식을 생성 및 수정하고 필요할 때 다시 읽는 시스템



LLM Wiki Mechanism

❖ STEP 1 – Source Ingestion: 원본 자료 입력

Drone_Design_Textbook.pdf
Drone_Databook.pdf
Paper_moter.txt
→ Text 내용 추출

➔ 즉 LLM이 읽을 수 있는 Source Content로 변환

❖ STEP 2 – LLM Knowledge Extraction: LLM이 원본을 분석

Wiki를 만들기 위한 prompt를 LLM에게 준다 →

다음 문서에서 중요한 정보를 추출하라.
1. 주요 Entity
2. 주요 Concept
3. 주요 Parameter
4. 중요한 주장이나 사실
5. 다른 개념과 연결될 수 있는 관계
6. Source에서 근거가 되는 내용

그리고 Source인 Text를 같이 준다

❖ STEP 3 – Existing Wiki Search: 기존 Wiki와 비교

Propeller
Motor
Payload
Hexacopter
→ 왼쪽 정보를 찾았고 이 내용이 있는지 기존 Wiki에서 찾기 →
/wiki/
Entities/
UAV.md
Motor.md
→ 정보가 없으면 새로 .md를 추가하고 .md 파일이 있으면 기존 .md에 추가

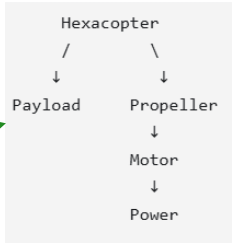
❖ SETP 4 – Knowledge Linking: Wiki page 연결

Propeller.md
Motor.md
Payload.md
Hexacopter.md
→ 여러 개의 .md 파일을 연결한다.

```
# Propeller

Related Topics:

- [[Motor]]
- [[Thrust]]
- [[Hexacopter]]
- [[Payload]]
```



❖ SETP 5 – User Query & Wiki Retrieval: 사용자가 Wiki를 실제로 사용

What is LLM Wiki

- LLM Wiki
- RAG vs LLM Wiki

LLM Wiki



종류	어디에 저장?	지속 여부	LLM Wiki 와 관계
Parameter Memory	Model Weight	Model 안에 영구적	거의 사용 안 함
Context Memory	Prompt/Context Window	현재 호출 동안만	사용
External Memory	.md, DB, Vector DB 등	영구적	**핵심**

RAG vs LLM Wiki

구분	RAG	LLM Wiki
기본 저장 단위	Chunk	Wiki Page / Concept / Entity
지식 생성 시점	질문할 때	문서 Ingest 할 때
원본 처리	Chunk 그대로 저장	LLM 이 읽고 지식으로 재구성
지속성	질문 후 결과가 보통 사라짐	지식이 Wiki 에 계속 남음
관계	Embedding 유사도 중심	Wiki Link / Graph / Semantic Search
새로운 문서	Vector DB 에 추가	기존 Wiki 까지 수정 가능
사람이 읽기	Chunk 는 어려움	Markdown Wiki 라 읽기 쉬움
Agent	없어도 됨	Agent 가 핵심적인 역할
Knowledge Graph	선택적	자연스럽게 만들기 쉬움
RAG 사용	핵심 구성요소	선택적으로 내부에서 사용 가능

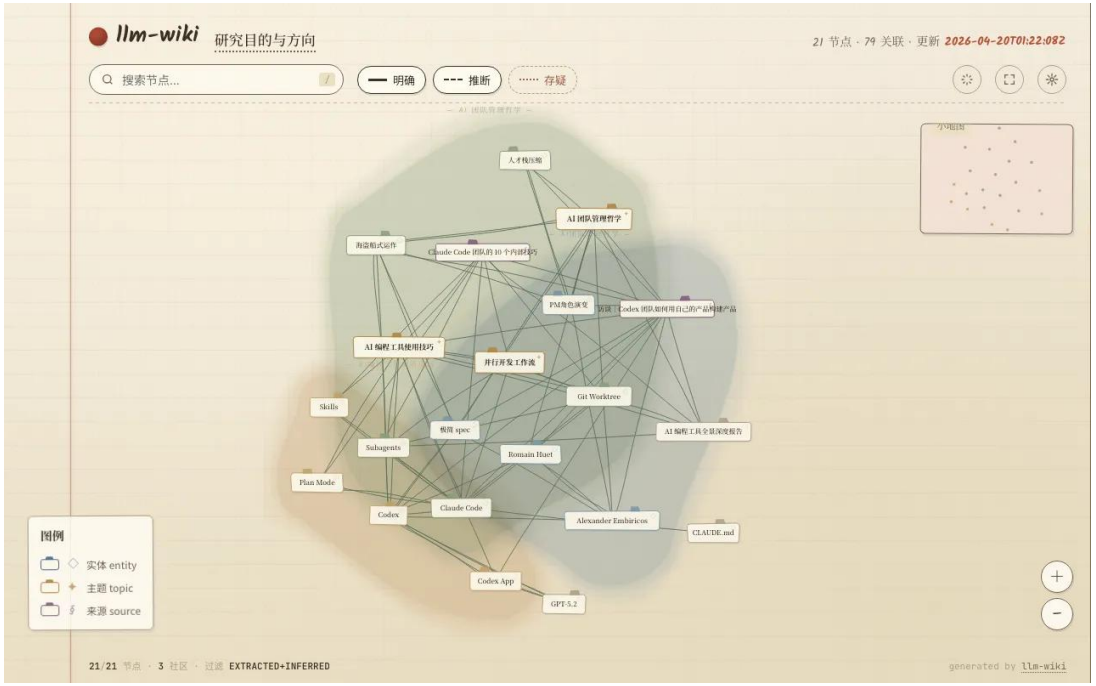
❖ RAG는 answer-time retrieval이고, Wiki는 durable knowledge를 지속적으로 유지하는 것

What is LLM Wiki

- LLM Wiki의 시각화
- LLM Wiki를 Drone databook에 활용하면 좋을까

Visualization

❖ 기존에 했던 Knowledge Graph처럼 시각화 된다



Drone databook

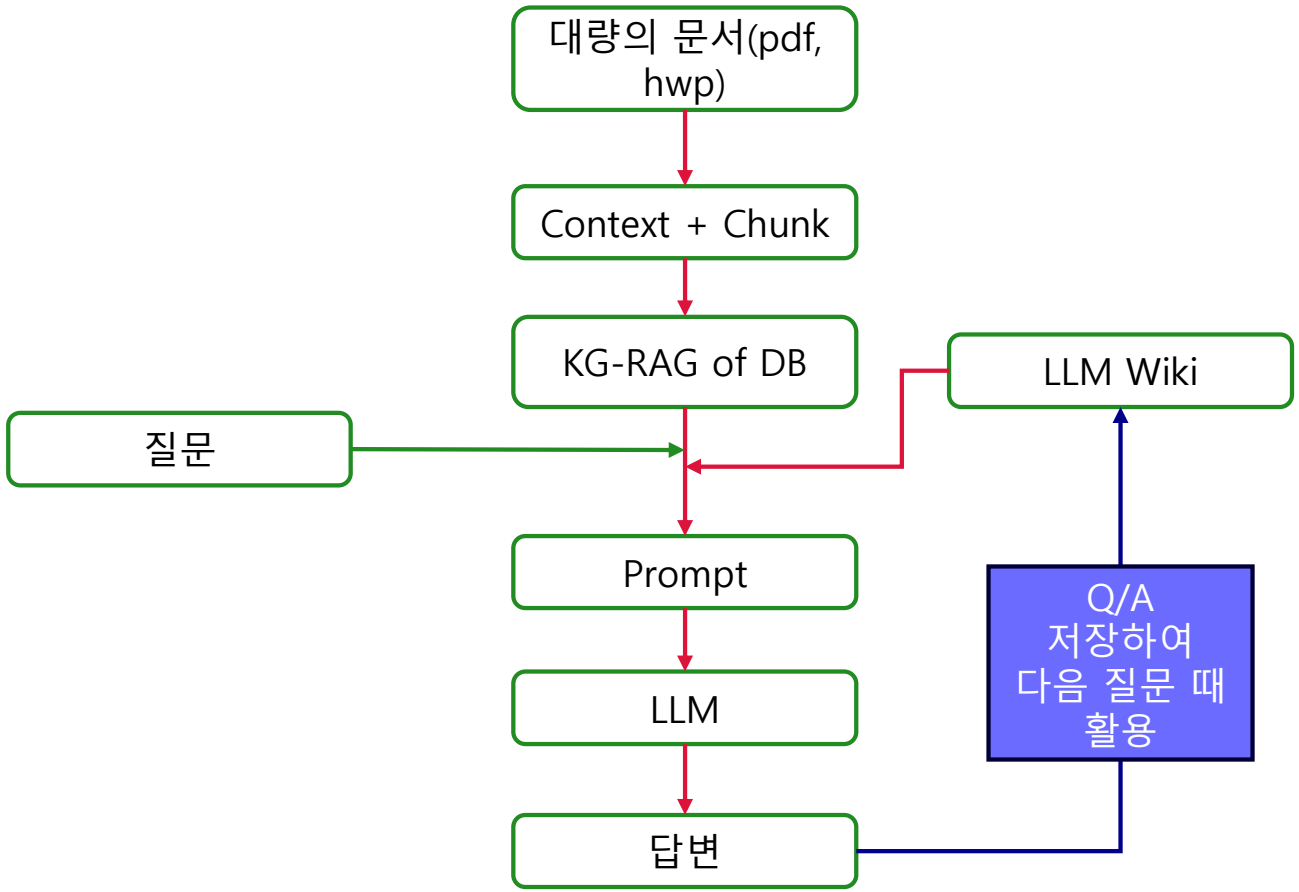
❖ 기존 KG-RAG를 활용

❖ 많은 양의 Drone textbook을 LLM Wiki로 사용하면 결국엔 RAG를 활용하여 검색해야 한다

❖ Drone을 설계한다면 설계 방향이 있을 것이고 그 방향을 LLM Wiki로 저장하여 LLM을 사용할 때마다 그 설계 방향으로 나아가는 방법으로 활용하면 좋을 것 같다

KG-RAG + LLM Wiki + (Context + Chunk)

workflow



예로 drone 설계 방향이 있을 경우에 그 방향에 대한 질문을 계속하고 답변을 받으면서 설계 방향성을 유지할 수 있다.

Any Questions?

이름 신 동 우

소속 한양대학교 기계공학과

이메일 sunwoongy@hanyang.ac.kr

Intelligent Design for Engineering Applications Lab

